

# Failure models in Gossip and Push-Sum

COP6539 Project 2, bonus · Ahmed Rageeb Ahsan, Saiful Islam

The simulator has no failure of any kind in the main report. This one adds two, each driven by a single parameter, and asks what each algorithm does as that parameter rises.

## Method

**death= $p$**  — each node independently dies at a random moment inside a 500 ms window, with probability  $p$ . A dead actor stops transmitting and discards whatever arrives, which models a machine that fails partway through a run.

**drop= $q$**  — each individual send is lost with probability  $q$ , independently. This models a link that is momentarily down rather than a node that is gone.

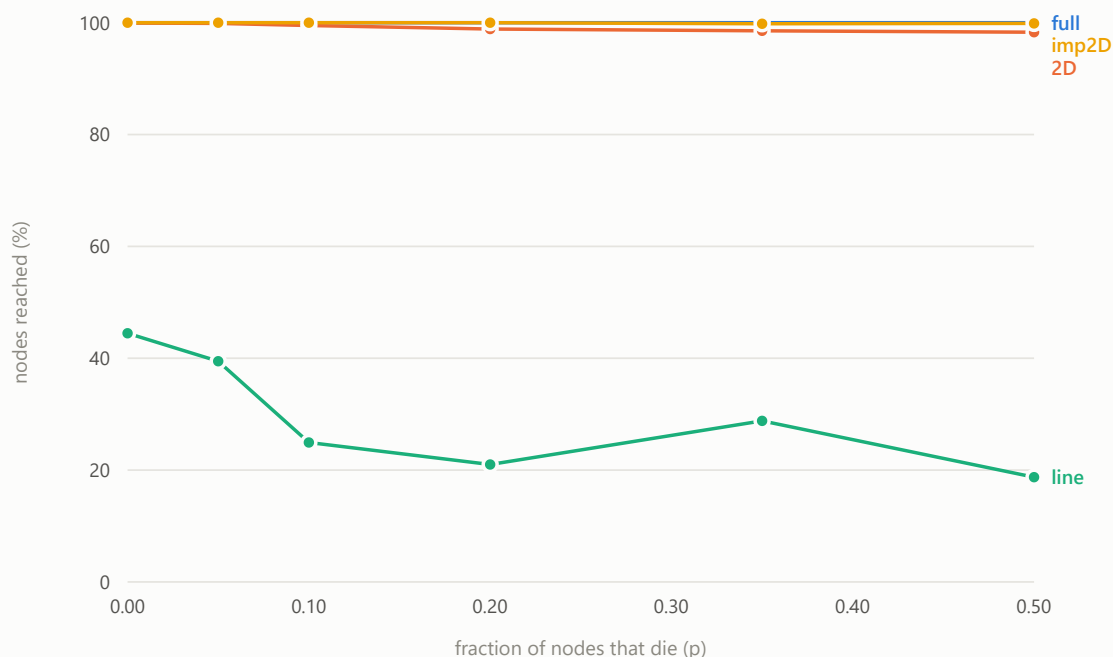
Network size is fixed at 500 nodes so the parameter is the only thing that varies, with three trials per point. The measure plotted is how much of the network the run actually reached — for Gossip the actors that heard the rumour, for Push-Sum the actors that met the termination rule. Under failure the interesting outcome is partial delivery, not slower delivery, so coverage is more informative than time.

## Results

■ full ■ 2D ■ line ■ imp2D

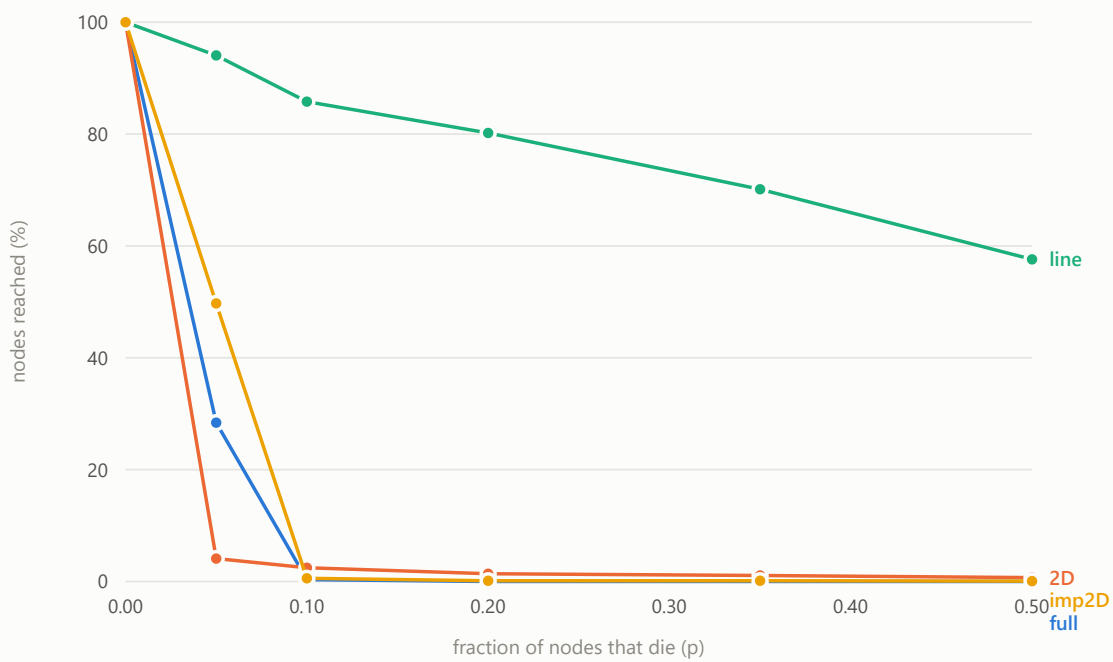
### Gossip: nodes reached under node failure

Mean over three trials, 500 nodes.



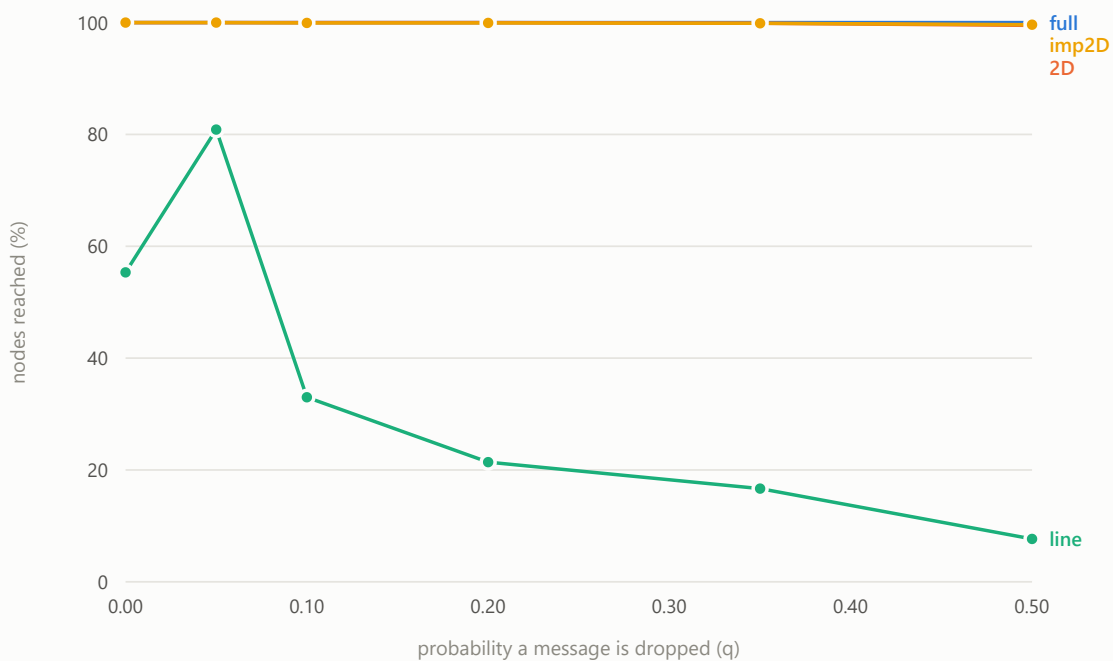
## Push-Sum: actors converged under node failure

Mean over three trials, 500 nodes.



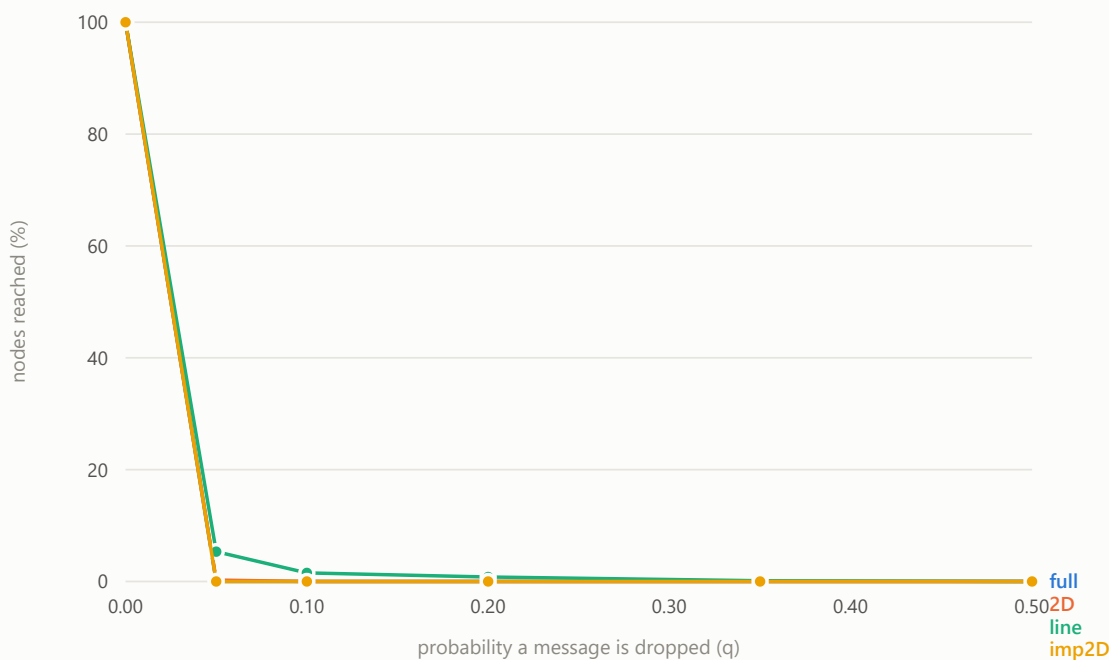
## Gossip: nodes reached under message loss

Mean over three trials, 500 nodes.



## Push-Sum: actors converged under message loss

Mean over three trials, 500 nodes.



## Findings

**The two algorithms fail in completely different ways, and the topology that protects one ruins the other.**

**Gossip under node death: connectivity is what buys resilience.** *full*, *2D* and *imp2D* are essentially untouched, holding above 98% even when half the nodes die, because every pair of survivors still has many routes between them. *line* falls from 44% to around 20%: a single dead node partitions a line, and by construction there is no second route. What matters is not how many nodes die but how many paths each death destroys.

**Push-Sum under node death inverts that ranking completely.** At just  $p = 0.05$ , *full* drops from 100% to **0%** — not one actor converges — while *line* still reaches 89%, and 54% even at  $p = 0.5$ . The reason is that Push-Sum's messages carry mass, and a dead actor absorbs whatever reaches it. On *full* a message can land on any node, so with a fraction  $p$  dead it is swallowed after about  $1/p$  hops; the circulating population evaporates and the network falls silent. On a *line* a message is confined between two neighbours, so the dead nodes merely cut the line into live segments inside which messages keep circulating.

**But terminating is not the same as being right.** Those surviving *line* actors converge on their own segment's average rather than the network's: relative error rises from  $1e-3$  with no failures to around  $5e-2$  at  $p = 0.05$ , roughly twenty times worse, while the actors that do finish on *full* with no failures are exact to  $2e-13$ . The curve counts actors that met the termination rule, not actors that met it correctly.

**Message loss separates the algorithms outright.** Gossip degrades gradually — a lost rumour costs nothing permanent, since the sender is still active and will send again. Push-Sum fails completely at even 5% loss on every topology, *full* included. That is not a tuning problem: Push-Sum's correctness

rests on mass conservation, so every dropped message destroys mass permanently, and with one message in flight per actor the population decays until nothing is left to carry the computation.

The practical reading: Gossip tolerates an unreliable network but needs a connected one; Push-Sum tolerates a sparse network but needs a reliable one. Surviving both would mean acknowledging the Push-Sum sends, buying reliability back at the cost of the asynchrony that makes it cheap.